Reference-Model-Driven Verification of MBIST and Self-Repair Logic: An Open-Source Case Study

Abhinav Kidambi
[TBD: affiliation]
[TBD: email]

***Abstract*—Memory built-in self-test (MBIST) and built-in self-repair (BISR) logic decide whether a defective memory is shipped, repaired or discarded, yet their RTL is often verified only by checking that the test finishes and that its pass/fail flag looks plausible. This paper presents a reference-model-driven method that checks every observable result of an MBIST/BISR block (pass/fail flag, each reported fail address and the final repair table) against a golden operation-level March model, under systematic memory fault injection. We apply the method to an open-source MBIST/BISR design taped out on a public multi-project shuttle, simulating 4,032 single-cell faults (stuck-at, transition and coupling) with open-source tools. The design's own testbench, which checks only completion, exposes none of the problems. Checking completion and the pass/fail flag against the golden model exposes 72 runs (2.8% of detectable faults) in which a faulty memory passes. The full scoreboard flags 100% of detectable faults: every reported fail address and every repair-table entry is wrong, and the faulty cell reaches the repair table in only 1.6% of failing runs. A comparison with full March C– also shows that the implemented shortened test detects only 50% of inter-word coupling faults. All designs, testbenches and scripts are open source.**

***Index Terms*—MBIST, built-in self-repair, March test, memory fault models, fault injection, design verification**

I. Introduction

Embedded memories dominate the area of modern SoCs, and nearly every memory instance is wrapped in MBIST and, increasingly, repair logic [1]. A bug in this logic is costly in both directions: a missed fault ships a defective part, while a wrong fail address or repair entry wastes redundancy or "repairs" a good cell and leaves the bad one in use.

In practice, MBIST RTL is frequently verified by running the test on a fault-free memory and checking that DONE rises and FAIL stays low, possibly with a few injected faults checked for FAIL. Such checks say little about the information that repair and diagnosis actually consume: *which* addresses failed, and *what* was written into the repair registers.

This paper makes the following contributions:

- A reference-model-driven verification method for MBIST/BISR RTL that predicts, for any injected fault, the pass/fail flag, the ordered list of fail addresses and the repair table, and scores the RTL against all three (Section III).
- A behavioural fault-injection testbench and campaign flow, built only on open-source tools, covering stuck-at (SAF), transition (TF) and idempotent coupling faults (CF).
- A case study on a taped-out open-source MBIST/BISR design (Section V) showing that completion-only and flag-only checks miss design bugs that the full scoreboard exposes in every detectable fault, together with root causes.
- [TBD: second design (row-redundancy MBIST) results and seeded-bug (mutation) campaign]

II. Background and Related Work

March tests and fault models. March tests apply a sequence of March elements, each a series of read/write operations applied to every address in ascending ($\Uparrow$), descending ($\Downarrow$) or either ($\Updownarrow$) order [2], [3]. March C–, $\{\Updownarrow\ (w0); \Uparrow (r0, w1); \Uparrow (r1, w0); \Downarrow (r0, w1); \Downarrow (r1, w0); \Updownarrow (r0)\}$, detects all SAFs, TFs, address-decoder faults and unlinked inter-word idempotent CFs in $10N$ operations [3]. Later work generated tests for linked faults [4], [5].

Verifying memory tests. Van de Goor and Smit automated the verification of March algorithms against fault models [6]; that work checks the *algorithm*, not the RTL that implements it. Assertion-based verification has been applied to March-based MBIST controllers [7], and a recent industrial methodology closes coverage gaps in shared-bus MBIST using physical-design-aware assertions [8].

Repair. BISR stores failing addresses and remaps accesses to spare rows, columns or words [1], [9]. Its correctness depends directly on the fail addresses reported by MBIST.

Our method differs from these works in checking, against a golden model, every result an MBIST/BISR block produces (flag, every fail address, repair table) under a systematic fault campaign, and in being fully reproducible with open-source designs and tools.

III. Method

Fig. 1 shows the flow. For each fault in a fault list, the testbench injects the fault into the behavioural memory, runs the MBIST to completion and records every fail report and the final repair table. Independently, a golden model applies the same March algorithm to an operation-level memory model with the same fault and predicts the correct results. A scoreboard compares the two and writes one row per fault.

A. Golden model

The golden model is an operation-level simulator of a $D \times W$ memory. A March algorithm is a list of (order,

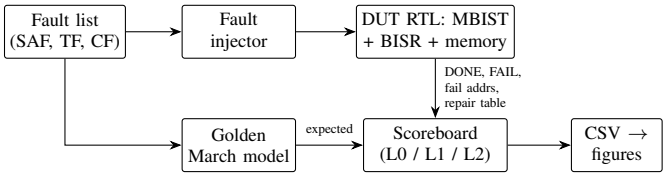


Fig. 1. Reference-model-driven verification flow.

operations) pairs. Each read compares every bit with the expected value; a mismatching read appends its address to the expected fail list. The expected repair table is the first R distinct failing addresses in detection order, where R is the number of repair entries. Spare rows are assumed fault-free and outside the tested address space, which is the documented intent of the case-study design. The model reproduces March theory: both the implemented test and full March C– detect all 512 SAFs and 512 TFs of a 32×8 memory.

B. Fault injection

Faults are injected behaviourally into the RTL memory array without editing the design. A correction is applied 1 ns after every clock edge, after any write has landed and before the next read samples the array: an SAF forces the cell to its stuck value; a TF reverts a forbidden $0 \to 1$ or $1 \to 0$ transition; an idempotent CF forces the victim to a value when the aggressor makes a given transition. The golden model implements the same semantics at the operation level.

C. Checker levels

To quantify what each level of checking can see, every run is scored at three levels: **L0**, DONE asserted (what the design's own test checks); **L1**, DONE and FAIL equal to the golden prediction; **L2**, the full scoreboard (FAIL, the ordered list of fail addresses and the repair table all equal the prediction). A run *exposes a design bug* at a level if that level's check fails.

IV. Experimental Setup

Design under test. We use Tiny Tapeout IHP 26a project #777 [10], an MBIST with a March C–style controller and a CAM-style BISR, at the taped-out commit (Apache-2.0). The RTL implements a 32×8 memory with two repair entries remapped to spare rows at addresses 30–31; the project documentation describes a 256×8 memory with 16 repair entries. The controller implements a shortened four-element test, $\{\Uparrow (w0); \Uparrow (r0, w1); \Downarrow (r1, w0); \Uparrow (r0)\}$, rather than the six-element March C–. [TBD: second DUT: row-redundancy MBIST [11]]

Fault list. All 32×8 cells for SAF0/1 and TF↑/↓ (1,024 faults), and idempotent CFs with physically adjacent aggressors (row above, row below, neighbouring bit in the same word), both transitions and both forced values (3,008 faults): 4,032 single-fault runs in total.

Tools. Icarus Verilog 12 [12]; Python for the golden model, campaign and plots. The full campaign runs in 14 s with 16 parallel simulations. The directed tests behind Table II were also reproduced on the EDA Playground web service (Icarus Verilog 12).

TABLE I
Fault campaign on TT #777 (single faults).

Class	Faults	Golden det.	DUT FAIL	Escapes	Correct*
SAF	512	512	504	8	0
TF	512	512	496	16	0
CF inter-word	1,984	992	960	32	0
CF intra-word	1,024	512	496	16	0
Total	4,032	2,528	2,456	72	0

*Detected runs in which every fail address and the repair table match the golden model.

V. Results

A. Detection

Table I and Fig. 3 summarise detection. The golden model predicts that the implemented test should detect 2,528 of the 4,032 faults; the DUT raises FAIL in 2,456. All 72 differences are escapes (a faulty memory passes); there are no false fails.

B. What each checker level sees

Fig. 4 shows, for the 2,528 detectable faults, how many runs expose a design bug at each level. The design's own test (L0) passes in every run. Comparing FAIL with the golden prediction (L1) exposes the 72 escapes, 2.8% of detectable faults. The full scoreboard (L2) exposes a bug in every detectable fault: in all 2,456 detected runs, the fail addresses and the repair table differ from the prediction. Of 2,712 fail reports that could be paired with a predicted report, only 280 (10.3%) carried the correct address; 2,416 were off by one. The faulty cell's address reached the repair table in only 40 of 2,456 failing runs (1.6%), and in every failing run both repair entries held the same address.

C. Root causes

Table II lists the bugs behind these results, each reproduced in simulation (B1–B3 traced in waveforms). Fig. 2 shows B1: the read of address 0 in the $\Downarrow (r1, w0)$ element returns a faulty value, but the element has already ended when the data arrives, so it is never compared.

D. Algorithm adequacy

Independently of the RTL bugs, the golden model quantifies the cost of the shortened algorithm. Full March C– detects all 1,984 inter-word CFs in our list; the implemented four-element test detects 992 (50%), because it omits $\Uparrow (r1, w0)$ and $\Downarrow (r0, w1)$. Both detect 50% of intra-word CFs, as expected for solid data backgrounds [2].

E. Second design and seeded bugs

[TBD: results on the row-redundancy MBIST [11]; seeded RTL-bug (mutation) campaign [13] comparing L0/L1/L2]

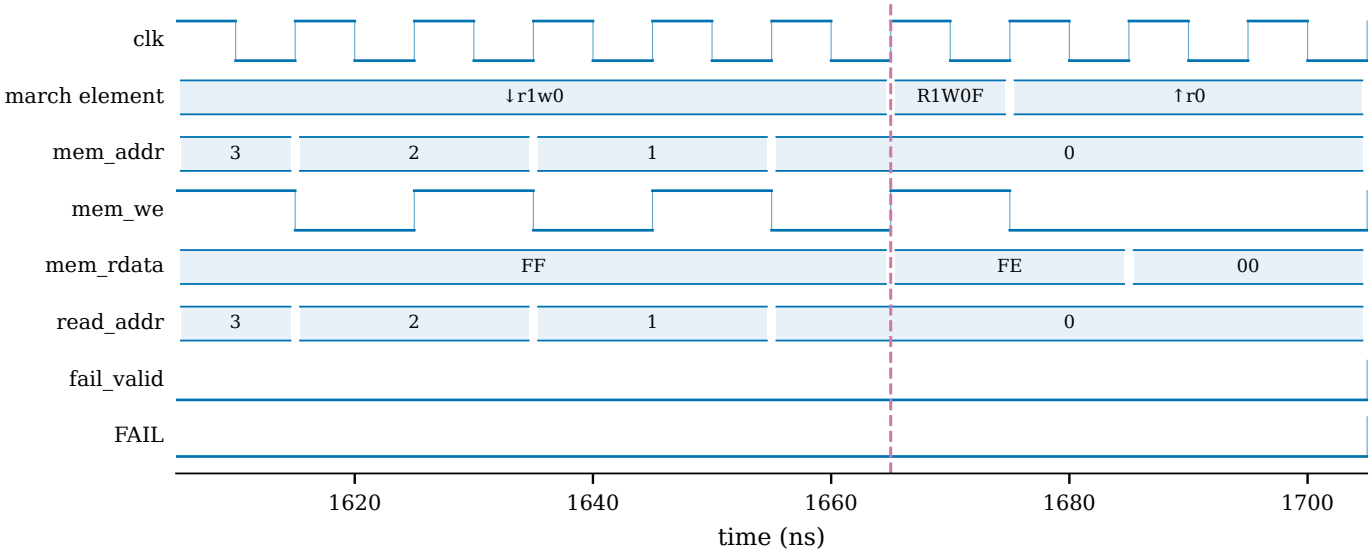


Fig. 2. B1: stuck-at-0 at address 0. The faulty read data (FE) arrives after $\Downarrow (r1, w0)$ ends and is never checked; FAIL stays low.

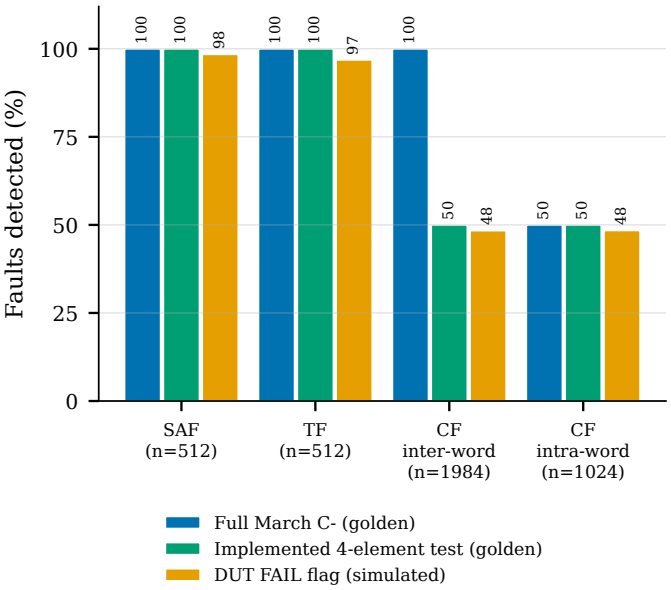


Fig. 3. Fault detection by class: full March C– and the implemented test (golden model) versus the DUT's FAIL flag.

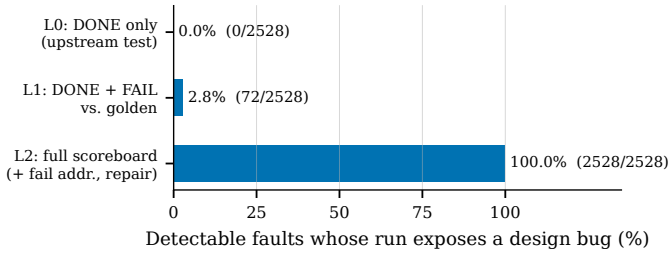


Fig. 4. Detectable faults whose run exposes a design bug, by checker level.

TABLE II
DESIGN BUGS FOUND IN TT #777.

ID	Symptom	Root cause
B1	Escapes at addresses 0 and 31 (72 runs)	Last read of $\Downarrow (r1, w0)$ and of final $\Uparrow (r0)$ never compared
B2	Fail address off by one	Compare uses read address after it has advanced
B3	One failure fills both repair entries	Non-blocking "inserted" flag read in the same cycle
B4	Second faulty cell never recorded	Table already full of duplicates (B3); seen with two injected faults
B5	Spares overlap user space	Spare rows 30–31 are also tested user addresses
B6	Docs vs. RTL mismatch	$256 \times 8/16$ entries documented; $32 \times 8/2$ implemented

VI. Discussion and Limitations

The study is simulation-only and uses small open-source memories; industrial MBIST has programmable algorithms, multiple data backgrounds and shared controllers. Fault injection is behavioural and single-fault, so multi-fault repair behaviour (B4) was examined only with directed tests. The golden model encodes an interpretation of the design's intent (spares outside the tested space, repair of the first R distinct failing addresses), which a different specification could change. The method itself is independent of these choices.

VII. Conclusion

Checking only completion and the pass/fail flag gives false confidence in MBIST and repair logic. On a taped-out open-source design, a golden-model scoreboard exposed escapes, wrong fail addresses and a broken repair table in every

detectable fault, none of which the design's own test detected. The designs, testbenches and scripts will be released with the paper. [TBD: repository link]

ACKNOWLEDGMENT

[TBD: AI disclosure wording to be finalised by the author.] Portions of the text, testbench code, scripts and figures were drafted with the assistance of an AI system (Claude, Anthropic) and were reviewed, verified and revised by the author, who takes full responsibility for the content.

REFERENCES

[1] Y. Zorian and S. Shoukourian, "Embedded-memory test and repair: Infrastructure IP for SoC yield," *IEEE Design & Test of Computers*, vol. 20, no. 3, pp. 58–66, 2003.

[2] A. J. van de Goor, *Testing Semiconductor Memories: Theory and Practice*. Chichester, U.K.: Wiley, 1991.

[3] A. J. van de Goor, "Using March tests to test SRAMs," *IEEE Design & Test of Computers*, vol. 10, no. 1, pp. 8–14, 1993.

[4] A. J. van de Goor, G. N. Gaydadjiev, V. N. Yarmolik, and V. G. Mikitjuk, "March LR: A test for realistic linked faults," in *Proc. IEEE VLSI Test Symp.*, 1996.

[5] S. M. Al-Harbi and S. K. Gupta, "Generating complete and optimal March tests for linked faults in memories," in *Proc. IEEE VLSI Test Symp.*, 2003.

[6] A. J. van de Goor and B. Smit, "Automating the verification of memory tests," in *Proc. IEEE VLSI Test Symp.*, 1994, pp. 312–318.

[7] A. Kumar, "Assertion based functional verification of March algorithm based MBIST controller," arXiv:2106.11461, 2021.

[8] S. Tulsyan *et al.*, "Physical design aware verification methodology for closing coverage gaps in SharedBus MBIST," 2026, IEEE Xplore doc. 11646504. [TBD: venue and pages]

[9] V. Sontakke and D. Atchina, "Memory built-in self-repair and correction for improving yield: A review," *Int. J. Electrical and Computer Engineering*, vol. 14, no. 1, pp. 140–156, 2024.

[10] Tiny Tapeout, "IHP 26a project #777: MBIST with MBISR," https://tinytapeout.com/chips/ttihp26a/777. [TBD: access date]

[11] D. Annayya, "mbist_ctrl," https://github.com/dineshannayya/mbist_ctrl. [TBD: access date]

[12] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog.

[13] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.